ChatGPT

Executive Summary

Modern GitHub Copilot offers an **agentic development workflow** in which multiple AI "agents" (Copilot coding agent, Copilot Chat/CLI, and even third-party LLMs) collaborate on tasks. The **Copilot coding agent** runs in the cloud (via GitHub Actions), autonomously editing code and opening pull requests in your repo. Other agents include local Copilot chat assistants in VS Code, the Copilot CLI, and integrations via Slack or Teams. By classifying tasks (e.g. bug fix, feature, refactor) and choosing the right agent – for example, a cloud agent for large, multi-file changes or a chat agent for quick fixes – teams can streamline development. A "hand-off" between agents (e.g. planning a task via Copilot Chat, then delegating to the cloud agent) is fully supported. This report presents a detailed design and step-by-step guide for a multi-agent Copilot workflow, including configuration examples, sample prompts, diagrams, and failure mitigation strategies.

Architecture and Components

An agentic Copilot workflow combines these core components: - **Cloud Agent (Copilot Coding Agent):** Runs on GitHub's servers (via GitHub Actions) and can autonomously edit multiple files and open PRs [1] [2]. It integrates tightly with GitHub (issues, branches, PRs) and is suited for well-defined tasks.
- **Local Chat Agents:** Copilot chat sessions in IDEs (VS Code, JetBrains, etc.) run locally and interactively [1]. These can be general conversational agents or specialized "plan" agents for outlining work [3].
- **Copilot CLI Agents:** The `gh` and `copilot` CLIs enable scripted and command-line access to Copilot. For example, `gh agent-task create "fix X"` starts a cloud agent task [4], and `copilot chat` or `copilot code` can run local assistants.
- **Third-Party & Custom Agents:** VS Code can invoke enabled third-party LLM agents (e.g. Anthropic Claude, OpenAI Codex) [5]. You can also define **Custom Agent Profiles** (personas or skill sets) in GitHub or IDE settings [6] [7] for tasks like "documentation writer" or "test generator".
- **Context Providers (MCP, Memory, Spaces):** Copilot uses the Model Context Protocol (MCP) to fetch code/text from your repo or the web [8]. Copilot Memory (if enabled) persistently stores user-specific info to improve continuity [9]. Copilot Spaces can hold long-term project context.

These agents communicate via GitHub or chat interfaces. For example, a **cloud agent session** appears in VS Code's Copilot Chat view [2], letting you monitor progress as it updates a PR. The **architecture** is shown in this simplified flowchart:

```
flowchart LR
  subgraph User Workflow
    U[Developer] -->|assigns task (issue, chat, CLI, etc.)| Decision{Task Type?}
    Decision -->|Feature/New Code| CloudAgent[Cloud Coding Agent (Copilot)]
    Decision -->|Quick Assist or Doc| LocalChat[Local Copilot Chat Agent]
    Decision -->|Scripted Task| CLI[Copilot CLI / GH CLI]
    LocalChat --> CloudAgent & LocalChat
```

```
    CLI --> CloudAgent
    CloudAgent -->|PR| Repo[GitHub Repo]
    Repo --> VSCode[Review in VS Code or GitHub]
    VSCode -->|Steer or Hand-off| CloudAgent
    VSCode -->|Chat/IDE| LocalChat
  end
```

In practice, tasks can be initiated from **GitHub Issues/PRs** (e.g. tagging an issue with "assign to @github"), from **VS Code Copilot Chat**, or via **Slack/Teams bots** that call Copilot. The agents use the repository code as context (via MCP) and push results back as new branches/PRs. Organizations configure **Copilot runners** (GitHub-hosted or self-hosted) to control where cloud agents execute [10].

Agent Selection & Decision Logic

Selecting the right agent depends on the task scope:

- **Cloud (Copilot coding) Agent:** Best for large, multi-file tasks and formal PRs. Trigger it via GitHub UI (Agents tab) or by `/task` in Copilot Chat (web or CLI) [11] [12]. It runs asynchronously and creates a pull request that you review [1] [13].
- **Local Copilot Chat Agent:** Best for interactive problem solving, brainstorming, or drafting. For example, use a local "Plan" agent to outline a design, then delegate to a cloud agent [3]. Local chat agents have access to open files and current editor context.
- **Copilot CLI / Scripting:** Useful for automation or when working in terminal/CI. The GitHub CLI's `gh agent-task` commands start and monitor cloud sessions [4]. The `copilot` CLI (which embeds Copilot Chat) can run background agents locally (e.g. research tools, custom tasks) [14].
- **Third-Party Agents:** If enabled, you can choose alternate LLM agents in VS Code. They may have different strengths (e.g. Claude's long context) but currently lack GitHub integration features [5].
- **Custom Agent Profiles:** If you need a specialized behavior (e.g. an "Accessibility Auditor" or "Documentation Expert"), define a custom agent with tailored prompts or tools [7]. You then select it when creating a session.

Decision logic can be partly automated by analyzing task metadata. For example, issue labels or keywords (e.g. "bug", "performance", "docs") might map to specific agents or prompt templates. A high-level workflow might look like:

1. **Classify Task:** Developer or system tags the task (bug, feature, test, docs, refactor, review).
2. **Choose Agent Type:** Based on classification, decide:
3. Bug/feature requiring code -> Cloud agent.
4. Quick refactor or doc -> Local chat agent.
5. Documentation request -> Custom doc-writing agent.
6. **Spawn/Configure Agent:** Start the selected agent (via GitHub UI, VS Code, CLI, etc.), optionally choosing a model or custom profile. Provide initial context (issue link, file snippet, etc.).
7. **Monitor and Steer:** As the agent runs, monitor logs (GitHub's Agents tab or CLI output) and provide steering prompts if needed [15].

8. **Switch if Needed:** If scope changes, hand off to a different agent. For example, after local planning you might `/delegate` to the cloud agent [16], or after cloud output you might open the session in VS Code for further editing [17].
9. **Review & Merge:** Finally, review the agent's pull request or code, request changes, and merge when approved [18].

Implementation Steps (By Environment)

Below are concrete steps for initiating and using Copilot agents in different environments. Each is fully supported by GitHub's documentation and tooling.

1. GitHub (Web UI) – Cloud Agent

1. **Open the Agents Panel:** In your repository on GitHub.com, click the "Agents" tab (or use the global Copilot icon) [11].
2. **Select Repository & Branch:** Ensure the correct repo is selected, and pick a base branch for the new work.
3. **Choose Agent & Model:** From the dropdown, select the **Copilot coding agent** (or a custom agent you created) [19]. Optionally pick an AI model (e.g. GPT-4). Third-party agents must be enabled in account settings first [5].
4. **Enter Prompt:** In the text box, describe the task in natural language. For example:

   ```
   Implement a user-friendly error message for file-not-found errors in the
   FileHandler module.
   ```

5. **Submit the Task:** Click **Start** or press Enter [20]. Copilot will kick off a session on GitHub's infrastructure. It immediately creates a draft pull request (tagged `[WIP]`) and begins pushing changes [21] [22].
6. **Monitor Progress:** On the Agents page, click the running session to see logs. You'll see the agent's "thoughts", tools used, and code diffs as it progresses [23]. Use the prompt box beneath the log to send steering instructions (one premium request per message) if the agent is veering off track [15].
7. **Review & Complete:** When the agent signals completion, it posts the final PR with a clear title/description and assigns you as reviewer [24]. Navigate to the PR, review the changes, and request revisions or merge as needed [18].

Note: You can also start a cloud task from Copilot Chat on GitHub by typing `/task <your prompt>` [12], or using GitHub CLI:

```
gh agent-task create "Fix bug in auth module to handle expired tokens"
```

which opens a PR behind the scenes [4].

2. Visual Studio Code (IDE Chat) – Local ⇄ Cloud

1. **Install Extensions:** Ensure you have the latest GitHub Copilot and GitHub Pull Requests extensions in VS Code.
2. **Start Chat:** Open the Copilot Chat view (e.g. using **Copilot: Chat**).
3. **Plan Locally (Optional):** For complex tasks, first use a local "Plan" agent to define steps. For example, ask "Plan how to refactor the authentication module." This interactive conversation helps clarify scope [3].
4. **Hand Off to Cloud:** Once ready, switch the session type to **Cloud** by using the session dropdown (or type `/delegate` in a background session) [25] [26]. This passes the entire chat context to a cloud agent session, which will continue execution on GitHub.
5. **Or Start New Cloud Session:** Alternatively, start a fresh cloud session by clicking **New Chat → Cloud** in the session list [27]. Choose Copilot or another cloud agent as above, then enter your prompt and submit.
6. **Monitor in VS Code:** The cloud session appears in the Chat view. You can watch its progress right in VS Code and send steering commands.
7. **Resume Locally:** When done (or anytime), use the "Open in VS Code" button in GitHub or the session actions dropdown:
8. **VS Code:** Click **Open in VS Code** in the cloud session details to launch a new VS Code window on that PR [28].
9. **CLI:** Choose **Continue in GitHub Copilot CLI** to copy a `copilot --resume=SESSION-ID` command [29], then paste and run it in your terminal to resume editing.
10. **Switch Agents Mid-Task:** You can switch agents anytime. For example, if the cloud agent completes initial work, you might open its branch locally and chat with Copilot to refine. Conversely, if a local chat hits its limit, delegate to cloud as above [1].

3. GitHub CLI (Terminal)

1. **Install CLI:** Use the GitHub CLI (v2.80.0+) which includes the `agent-task` commands (preview).
2. **Create Task:** Run:

   ```
   gh agent-task create
   "Refactor the data layer to use async calls and handle pagination"
   ```

   You can add flags: `--repo owner/repo`, `--base branch`, `--model gpt-4`, etc. The CLI prompts for confirmation and then starts the session [4].
3. **Follow Output:** Use `gh agent-task log --follow` to stream logs to your console. The CLI will show the agent's progress and final PR link.
4. **Continue Session Locally:** Once the PR is created, you can `gh pr checkout <number>` or use `copilot --resume=<SESSION-ID>` if you want to bring it into the Copilot CLI chat environment [29].
5. **Automation:** You can embed `gh agent-task` in shell scripts or CI pipelines. For example, a GitHub Actions workflow could automatically classify new issues and invoke `gh agent-task create`.

4. Slack / Teams Integration

1. **Install GitHub Bot:** Add the official **GitHub** app in your Slack workspace or Teams channel (one-time setup).
2. **Authenticate:** In Slack/Teams, DM or mention the bot and authorize it to your GitHub account and set a default repository (via the interactive prompts) [30] [31].
3. **Ask Copilot in Chat:** In any channel or thread, mention the bot and your request. For example:

   ```
   @GitHub Copilot Add user-friendly error messages in the README for all commands, and open a pull request.
   ```

   (Or in Slack, the syntax `@GitHub Add "Hello" to README in repo=owner/repo branch=main` [32].)
4. **Review Bot Response:** Copilot will reply with a confirmation and link to the PR it opened [33]. Use Slack/Teams thread messages as **context**, so follow-up messages refine the task if needed.
5. **Issue Creation:** You can also ask Copilot to draft issues in Slack/Teams by describing a feature or bug. The bot will use the thread history as context and then present a draft issue for review [34].
6. **Follow Up:** Once Copilot finishes a PR, GitHub and the integration will notify you of the new PR. Continue working by reviewing it on GitHub or in your IDE.

5. Other Tools

- **Raycast (macOS/Windows):** Install the GitHub Copilot extension and use the "Create Task" command to start sessions similarly to the CLI [35].
- **Copilot MCP (IDE Plugins):** Some IDEs or tools (with Model Context Protocol) let you run Copilot tasks by enabling a `create_pull_request_with_copilot` tool [36].

Sample Configurations and Prompts

Below are examples of configuration snippets, prompt templates, and policies for agent usage:

- **GitHub Actions (Custom Runners):** In `.github/workflows/copilot-setup-steps.yml`, you can specify runner labels. For example:

  ```yaml
  on: [push]
  jobs:
    copilot-setup:
      runs-on: ubuntu-latest
      steps:
        - name: Checkout
          uses: actions/checkout@v3
        # Other setup steps...
  ```

  This file can override organization defaults [37] [38].

- **VS Code Settings (settings.json):** Ensure Copilot is enabled and allow agent mode:

```
{
  "github.copilot.enable": true,
  "github.copilot.experimental.suggestionsHistory": true,
  "github.copilot.cloudAgents": ["copilot", "claude"]
}
```

- **Copilot CLI Configuration (~/.config/copilot/config.json):** You can set defaults such as model or memory:

```
{
  "model": "gpt-4",
  "memory": {"enabled": true},
  "tools": ["shell", "filesystem"]
}
```

- **Prompt Templates:** For agent assignment, tailor prompts to task type:
- *Bug Fix:* "Fix the bug in `app/controller.js` where submitting the form causes a 500 error. Explain the fix in comments."
- *Feature:* "Implement a REST API endpoint `GET /api/users/search` that searches users by name. Write tests as well."
- *Refactor:* "Refactor the `DataProcessor` class to improve performance. Use async/await and reduce database calls."
- *Code Review:* "Review the attached PR for security issues and suggest improvements in comments."
- **Agent Profiles:** Example `agent.yaml` for a custom "Docs Writer" agent:

```
name: "Docs Writer"
instructions: "You are a documentation specialist. Provide clear, concise
docs."
tools: ["web_search", "code_index"]
example_tasks:
  - "Write a README for the authentication module."
  - "Document the new caching API."
```

- **Policies and Permissions:**
- *Branch Protections:* Ensure only authorized agents or users can merge (e.g. require review on PRs).
- *Tool Permissions:* Cloud agents run in a sandbox (no internet). If using Copilot CLI agents with tools, handle permissions via the copilot SDK hooks (prompting user) [39].
- *Memory/Data:* Enable Copilot Memory in settings for plans/notes across sessions (preview feature) [9].
- *Security:* Limit sensitive code by using the *Content Exclusion* policy in GitHub or by disabling web access for agents [40] [41].

Example Scenarios

Below are concrete examples of tasks with commands, prompts, and expected agent behavior:

1. **Bug Fix:**
2. *Context:* Issue titled “Crash on Save”. Developer in Slack says: `@GitHub Fix the null-pointer bug in` saveDocument() `.`
3. *Agent Trigger:* Slack Copilot bot picks up the mention [32].
4. *Prompt Used:* “In the repo octo-org/app, fix the null-pointer exception in `DocumentController.save()` that occurs when no title is provided.”
5. *Expected Behavior:* Cloud agent opens a draft PR, finds where `save()` is called without a title, adds a check or default value, writes comments. When done, bot replies with PR link [33].
6. **Feature Development:**
7. *Context:* In VS Code, developer chats: “Plan the steps to add a `POST /api/notifications` endpoint with JSON body validation.”
8. *Local Agent:* A Plan agent outlines tasks. Developer then types `/delegate` to send the plan to cloud [26].
9. *Cloud Agent:* Given the plan, it implements the endpoint, schema checks, and opens PR.
10. *Review:* Developer reviews PR in VS Code (Open in VS Code command) [28], requests one change (“add error handling”), then types a steering prompt in the agent session. Agent adjusts and pushes update.
11. **Refactor:**
12. *Context:* GitHub issue "Refactor payment module". Developer uses GitHub CLI:

    ```
    gh agent-task create "Refactor the payment processing code to use the new PaymentService class for all transactions."
    ```

13. *Agent Behavior:* The CLI begins a Copilot session and eventually outputs a PR link. The cloud agent updates files to use `PaymentService`, removes deprecated code, and notes assumptions in PR description [22].
14. **Code Review:**
15. *Context:* A teammate opens a PR and wants Copilot’s input. In GitHub Chat: `/task Review changes in PR#42 and suggest performance improvements.`
16. *Agent Behavior:* The coding agent reads the PR diff via MCP tools and comments with specific suggestions (e.g. “This loop can be optimized by caching”) and possibly pushes minor optimizations as a new commit to the branch.
17. **Documentation:**
18. *Context:* Developer asks Copilot Chat in VS Code: “Describe what `processOrder()` does and draft a usage example.”
19. *Local Agent:* A local Copilot chat agent analyzes the code and generates a doc comment and example. No PR is opened (this is a local assistant scenario).
20. **Tests:**
21. *Context:* From the GitHub UI Agents tab, user selects “Create Task” and enters: “Generate unit tests for all functions in `math_utils.py`.”

22. *Agent Behavior:* Cloud agent adds a new test file `test_math_utils.py` with test cases for each function, and opens a PR with the tests [22].

Agent Types, Triggers, Pros/Cons

Agent Type	Invocation/Trigger	Pros	Cons
Cloud (Copilot coding)	GitHub Agents UI, `/task` in Chat, `gh agent-task`, Slack/ Teams mention [11] [12]	Handles multi-file tasks, uses full repo context, auto PR generation [1] [22]	Higher latency (minutes), limited to repo context (no local runtime) [42]; consumes Copilot credits
Local Copilot Chat	VS Code chat (Copilot/ Plan agents)	Instant feedback, uses local file context, interactive loop [1]	Cannot autonomously commit code (manual copy/Paste needed); limited context window
Copilot CLI	`gh agent-task`, `copilot chat/code`	Scriptable; can run in CI; integrates with shell and tools [4] [43]	Text-only interface (no GUI); context limited to command history
Third-party Agent (e.g. Claude)	VS Code cloud session (needs enabling)	Potentially longer context, different model strengths [5]	Fewer GitHub integrations; may require separate subscription
Custom Agent Profile	Selected in GitHub or VS Code during session	Tailored persona/skills; can include specialized tools [7]	Requires setup effort; complexity in tuning instructions

Timeline of Agent Switching

```
flowchart LR
  User([Developer]) --> IdentifyTask[Identify and describe task]
  IdentifyTask --> VSChat[Use Copilot Chat locally]
  VSChat --> PlanAgent[Local Plan Agent outlines task]
  PlanAgent --> Delegation[Delegate to Cloud Agent (/delegate)]
  Delegation --> CloudAgent["Copilot Cloud Agent runs (GitHub Action)"]
  CloudAgent --> Review[Review PR in GitHub/VSCode]
  Review -->|Needs more work| VSChat
  Review -->|Approve| Merge[Merge pull request]
```

This flow shows a typical agent-switch: user discusses with a local Plan agent, delegates to the cloud agent for execution, then reviews/merges.

Failure Modes & Mitigations

- **Off-Topic or Unsafe Output:** An agent may misunderstand a vague prompt. *Mitigation:* Provide clear, step-by-step instructions or examples. Use steering prompts immediately if it goes off course [15].
- **Overgeneration / Large Scope:** A broad task might be partially done or incomplete. *Mitigation:* Break into smaller tasks or refine prompt. Use the Plan agent first to outline scope.
- **Slow Response / Timeouts:** Cloud sessions can take minutes for big tasks. *Mitigation:* Monitor progress; if stalled, stop the session and try a simpler prompt. For time-critical tasks, use a local chat agent or Copilot CLI.
- **Merge Conflicts or Environment Issues:** The agent's PR might conflict or have build issues. *Mitigation:* Review code early; possibly open the session in VS Code and run tests locally. Use Copilot's built-in checks or CI to catch errors.
- **Authentication/Access Errors:** Slack/Teams bot requires proper GitHub permissions. *Mitigation:* Ensure the GitHub App for Slack/Teams is installed and you have write access to the target repo [44].
- **Token or Rate Limits:** Copilot usage is metered. *Mitigation:* Use agent features judiciously; monitor consumption on GitHub's usage metrics dashboards [45] [18].

Performance and Latency

- **Cloud Agents:** Typically complete small tasks (a few files) in seconds to minutes, but large refactors may take longer (up to tens of minutes) due to the build-run-test cycle on GitHub Actions [22].
- **Local Agents:** Instant response (sub-second to seconds) but limited by model context.
- **Copilot CLI:** Roughly similar to cloud agents for tasks (since it uses the same backend), but can follow logs live in the terminal.
- **Parallel Execution:** You can run multiple cloud sessions concurrently (on separate branches) to speed up overall throughput.
- **Latency Tips:** Using repository indexing (MCP) and Copilot Memory can reduce time the agent spends fetching context [46]. Selecting a faster model (if available) can also help.

Recommended Tools & Resources

- **Official Docs:** GitHub Copilot documentation (Agents) [11] [22] and Microsoft Copilot docs [47] [25] are the primary references. Prioritize these when setting up.
- **GitHub CLI:** Use the latest GitHub CLI (`gh agent-task`) [4].
- **VS Code Copilot Extension:** Ensure you have GitHub Copilot extension and GitHub Pull Requests.
- **Copilot CLI:** Available via `npm i -g @githubnext/copilot-cli`. Supports background agents and session resumption [48].
- **Integrations:** Set up Slack or Teams by installing the GitHub app and authorizing Copilot access. Raycast extension for Copilot is another helpful UI.
- **Copilot Memory & MCP:** Enable experimental features (Memory, Copilot Spaces) in your Copilot account for improved context handling [9] [49].
- **Enterprise Admin:** Use the Copilot admin settings to configure runners and permissions [10].
- **Community & Support:** Refer to the GitHub Copilot community forums and the GitHub Copilot *Chat Cookbook* for prompt examples.

Migration Checklist & Best Practices

- **Enable Copilot Agents:** Make sure your organization has Copilot Pro/Enterprise and the cloud agent is enabled in settings [11].
- **Set Up Permissions:** Install the GitHub App for Slack/Teams and grant it write access to target repos [30] [50]. Ensure branch protections allow PRs from bots.
- **Configure Runners:** Choose appropriate GitHub-hosted or self-hosted runners for cloud agents in org settings [10].
- **Define Custom Agents:** Create any needed custom agent profiles early (e.g. "Bug Fix Assistant") so team members can select them [7].
- **Train the Team:** Educate developers on using `/task`, chat hand-offs, and reviewing agent PRs. Start with low-risk tasks (docs, tests) before core code.
- **Review and Iterate:** Always review agent-produced code. Use agents to accelerate *your* development, not replace oversight. Merge only after human approval [24].
- **Measure & Refine:** Monitor Copilot usage metrics and agent session logs. Adjust prompts and configurations based on what works (see **GitHub Copilot usage metrics** docs).
- **Update Continuously:** Agent tooling is evolving. Regularly check GitHub's official docs (last updated April 2026) for new features like improved model selection or streaming APIs [20] [25].

By following these guidelines – classifying tasks, invoking the right agent, and using built-in hand-offs – teams can leverage GitHub Copilot's full agentic workflow to reduce manual effort, while maintaining code quality through continuous review and best practices [21] [18].

---

[1] [2] [3] [5] [6] [16] [25] [26] [27] [42] [47] Cloud agents in Visual Studio Code
https://code.visualstudio.com/docs/copilot/agents/cloud-agents

[4] [12] [35] [36] Asking GitHub Copilot to create a pull request - GitHub Docs
https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/create-a-pr

[7] Creating custom agents for Copilot cloud agent - GitHub Docs
https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/create-custom-agents

[8] [14] [39] [43] [48] GitHub Copilot Agents | Microsoft Learn
https://learn.microsoft.com/en-us/agent-framework/agents/providers/github-copilot

[9] [46] [49] About GitHub Copilot cloud agent - GitHub Docs
https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent

[10] [37] [38] [40] Configuring runners for GitHub Copilot cloud agent in your organization - GitHub Docs
https://docs.github.com/en/copilot/how-tos/administer-copilot/manage-for-organization/configure-runner-for-coding-agent

[11] [13] [15] [17] [18] [19] [20] [22] [23] [28] [29] Managing cloud agents - GitHub Docs
https://docs.github.com/en/copilot/how-tos/use-copilot-agents/manage-agents

[21] [24] GitHub Copilot coding agent 101: Getting started with agentic workflows on GitHub - The GitHub Blog
https://github.blog/ai-and-ml/github-copilot/github-copilot-coding-agent-101-getting-started-with-agentic-workflows-on-github/

[30] [32] [33] [34] [41] [44] [45] Integrating Copilot cloud agent with Slack - GitHub Docs
https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/integrate-cloud-agent-with-slack

31 50 Integrating Copilot cloud agent with Teams - GitHub Docs

https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/integrate-cloud-agent-with-teams